

Cheat sheet HTTP, API, Fetch e Axios

Obiettivo: ripassare solo le cose essenziali per la verifica Boolean.

Questi argomenti sono stati introdotti alla fine del percorso prima della verifica, quindi e' importante capirli a livello pratico e concettuale, senza andare troppo in profondita'.

Idea generale

Quando una pagina web ha bisogno di dati esterni, per esempio utenti, prodotti, post o immagini, il browser fa una richiesta a un server.

Il flusso base e':

1. Il client, cioe' il browser, fa una richiesta HTTP.
2. Il server riceve la richiesta.
3. Il server prepara una risposta.
4. Il client riceve i dati.
5. JavaScript usa quei dati, per esempio stampandoli in console o renderizzandoli nel DOM.

Parole chiave

Termine	Significato semplice
Client	Chi fa la richiesta. Nel nostro caso spesso e' il browser.
Server	Chi riceve la richiesta e restituisce una risposta.
HTTP	Protocollo usato per comunicare tra client e server sul web.
Request	Richiesta inviata dal client al server.
Response	Risposta inviata dal server al client.
API	Interfaccia che espone dati o operazioni utilizzabili da altri programmi.
Endpoint	URL specifico di una API, per esempio <code>/users</code> o <code>/posts/1</code> .
AJAX	Tecnica per fare richieste asincrone da JavaScript senza ricaricare tutta la pagina.
Fetch API	API nativa del browser per fare richieste HTTP da JavaScript.
Axios	Libreria esterna che semplifica le richieste HTTP.
JSON	Formato testuale molto usato per scambiare dati tra client e server.
Query params	Parametri aggiunti all'URL dopo <code>?</code> , per esempio <code>?name=mario&page=2</code> .
Postman	Strumento per testare API senza scrivere codice nel browser.

HTTP in breve

HTTP e' il protocollo che permette a client e server di comunicare.

Una richiesta HTTP puo' contenere:

- metodo, per esempio **GET** o **POST**;
- URL o endpoint;
- headers, cioe' informazioni aggiuntive;
- body, cioe' dati inviati al server, soprattutto in richieste come **POST**.

Una risposta HTTP contiene:

- status code;
- headers;
- body, cioe' il contenuto della risposta.

Metodi HTTP essenziali

Metodo	Uso principale
GET	Recuperare dati da una risorsa.
POST	Inviare dati per creare una nuova risorsa o mandare informazioni al server.

Per la verifica probabilmente basta saper fare e spiegare una richiesta **GET**.

Nota utile: un form HTML senza attributo **method** usa **GET** di default. Se scriviamo **method="post"**, allora invia una richiesta **POST**.

Status code essenziali

Codice	Significato
200 OK	La richiesta e' andata bene.
201 Created	Una risorsa e' stata creata.
301 Moved Permanently	Redirect permanente.
302 Found	Redirect temporaneo.
400 Bad Request	Il client ha mandato una richiesta non valida.
401 Unauthorized	Serve autenticazione.
403 Forbidden	Il client e' riconosciuto, ma non ha permesso.
404 Not Found	Risorsa non trovata.
500 Internal Server Error	Errore lato server.

Query params

I query params sono parametri scritti nell'URL.

Esempio:

```
https://example.com/users?name=mario&active=true
```

Significa:

- endpoint: `https://example.com/users;`
- primo parametro: `name=mario;`
- secondo parametro: `active=true;`
- `?` inizia la query string;
- `&` separa piu' parametri.

I query params non fanno parte del body: fanno parte dell'URL.

Una richiesta `GET` puo' avere query params.

JSON

JSON significa JavaScript Object Notation.

E' un formato testuale usato per scambiare dati.

Esempio JSON:

```
{  
  "name": "Mario",  
  "age": 30,  
  "active": true  
}
```

In JavaScript:

- `JSON.parse()` trasforma una stringa JSON in dati JavaScript;
- `JSON.stringify()` trasforma dati JavaScript in una stringa JSON.

Con `fetch`, spesso usiamo `response.json()` per leggere il body della risposta come JSON.

Asincronia: perche' usiamo `.then()`

Una chiamata HTTP richiede tempo.

JavaScript non puo' fermare tutta la pagina mentre aspetta il server. Per questo la richiesta e' asincrona.

`fetch()` non restituisce subito i dati finali: restituisce una Promise.

Una Promise rappresenta un valore che arrivera' in futuro.

Per usare quel valore dobbiamo lavorare dentro `.then()`.

Esempio concettuale:

```
fetch(url)
  .then(function (response) {
    // qui la risposta HTTP e' arrivata
  });
```

Il codice fuori da `.then()` parte subito, prima che i dati siano arrivati.

Per questo questo codice non funziona come ci aspettiamo:

```
let users;

fetch(url).then(function (response) {
  users = response;
});

console.log(users); // probabilmente undefined
```

La regola pratica e': se ti servono i dati della chiamata, lavora dentro `.then()` o dentro funzioni chiamate da `.then()`.

Fetch API: esempio base

```
const url = 'https://jsonplaceholder.typicode.com/users';

fetch(url)
  .then(function (response) {
    return response.json();
  })
  .then(function (users) {
    console.log(users);
  })
  .catch(function (error) {
    console.error('Errore nella richiesta:', error);
  });
```

Perche' ci sono due `.then()`?

1. Il primo riceve la risposta HTTP.
2. `response.json()` legge e converte il body in dati JavaScript.
3. Il secondo riceve i dati pronti da usare.

Axios: esempio base

Axios e' una libreria esterna. Nel browser si puo' includere con una CDN.

```
<script src="https://cdn.jsdelivr.net/npm/axios/dist/axios.min.js">
</script>
```

Esempio:

```
const url = 'https://jsonplaceholder.typicode.com/users';

axios.get(url)
  .then(function (response) {
    console.log(response.data);
  })
  .catch(function (error) {
    console.error('Errore nella richiesta:', error);
  })
  .finally(function () {
    console.log('Richiesta completata');
  });
```

Con Axios i dati sono spesso già disponibili in `response.data`.

Con Fetch invece dobbiamo chiamare `response.json()`.

Renderizzare dati nel DOM

Esempio minimale con Axios:

```
<ul id="users-list"></ul>
```

```
const listElement = document.getElementById('users-list');
const url = 'https://jsonplaceholder.typicode.com/users';

axios.get(url)
  .then(function (response) {
    const users = response.data;

    listElement.innerHTML = users.map(function (user) {
      return `<li>${user.name}</li>`;
    }).join('');
  })
  .catch(function (error) {
    console.error('Errore nella richiesta:', error);
  });
```

Idea importante:

1. Faccio la richiesta.
2. Aspetto i dati dentro `.then()`.
3. Uso i dati per creare HTML.
4. Inserisco l'HTML nella pagina.

Fetch vs Axios

Fetch	Axios
Integrato nel browser.	Libreria esterna.
Richiede <code>response.json()</code> .	I dati sono in <code>response.data</code> .
Va bene per iniziare e capire il meccanismo.	Spesso piu' comodo nella pratica.

Per la verifica: saper spiegare entrambi, ma saper scrivere almeno una `GET` con Axios o Fetch.

Cosa dire alla verifica

Risposta breve possibile:

Una API e' un'interfaccia che permette al nostro codice di comunicare con un altro software. Nel web spesso usiamo API esposte tramite endpoint HTTP. Con JavaScript possiamo fare chiamate AJAX usando Fetch o Axios. La richiesta e' asincrona, quindi i dati non sono disponibili subito: dobbiamo usarli dentro `.then()`, dove riceviamo la risposta e possiamo stamparla in console o renderizzarla nel DOM.

Checklist minima

Prima della verifica assicurati di saper:

- spiegare client, server, request e response;
- spiegare cos'e' una API;
- spiegare cos'e' un endpoint;
- dire a cosa serve una richiesta `GET`;
- riconoscere query params in un URL;
- spiegare cos'e' JSON;
- fare una `fetch(url).then(...).catch(...)`;
- fare una `axios.get(url).then(...).catch(...)`;
- dire perche' i dati vanno usati dentro `.then()`;
- stampare il risultato in console con `console.log()`;
- renderizzare una lista semplice nel DOM.